

Sistema de Microservicios del Aeropuerto Internacional Jorge Chávez

Hito 1 y Hito 2 — avance y entrega

Benjamin Toro — Líder / Arquitectura

Guillermo — MS1 Pasajeros

Mariano — MS2 Vuelos

Edinson — MS3 Infraestructura

Fabricio — MS4/MS5 + Data Science

Alexander Leon — Frontend

Recorrido del proyecto

1

Introducción y objetivos

2

Arquitectura y diagrama

3

Backend — 5 microservicios

4

Transformaciones del
modelo

5

Frontend

6

Data Science — ingesta,
Glue, Athena

7

Despliegue y seguridad

8

Del Hito 1 al Hito 2

9

Repositorios, conclusiones y
demo

Introducción y objetivos

Objetivo y alcance

Modernizar el modelo de datos del Aeropuerto Jorge Chávez (Base de Datos I), hoy monolítico, descomponiéndolo en 5 microservicios sobre AWS, con frontend web SPA, ingesta de datos y analítica gestionada (Glue + Athena).

Enfoque: demostrar cada requisito con evidencia real (curl, COUNT(), logs), no solo en el papel.*

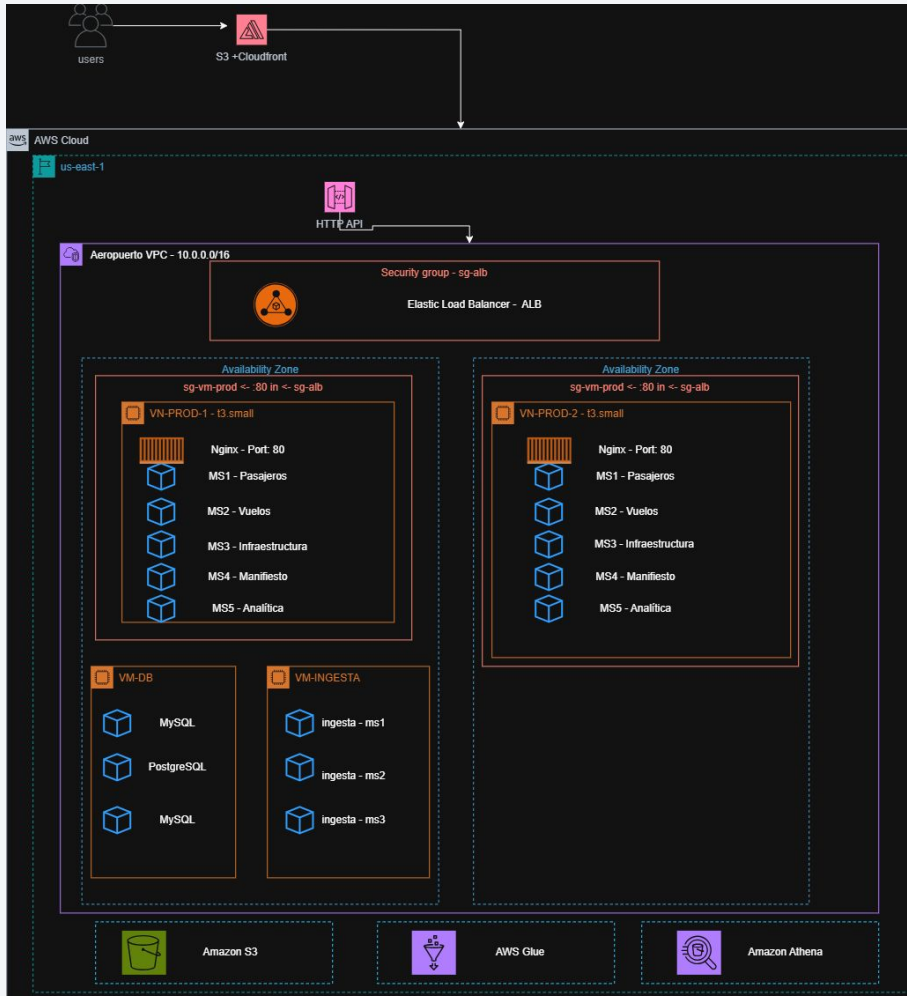
Requisitos técnicos clave

- 3 lenguajes + 3 bases de datos distintas (2 SQL + 1 NoSQL)
- 1 microservicio consume a otro · 1 sin BD · 1 analítico con Athena
- Carga masiva $\geq 20\,000$ registros por BD
- 2 VMs de producción + balanceador privado + API Gateway HTTPS
- Frontend consumiendo los 5 microservicios (≥ 2 métodos c/u)
- Pipeline de Data Science: ingesta $\rightarrow$ S3 $\rightarrow$ Glue $\rightarrow$ Athena

Arquitectura de la solución

Red	VPC aeropuerto (10.0.0.0/16), 2 subredes públicas + 2 privadas, 2 AZ. NAT Gateway como única salida de las subredes privadas.
Borde	Única entrada pública: Amazon API Gateway (HTTPS) → VPC Link → ALB interno.
Cómputo	2 VM-PROD idénticas (nginx + MS1..MS5) detrás del ALB; VM-DB (MySQL + PostgreSQL + MongoDB) sin IP pública.
Datos	VM-INGESTA copia los datos operacionales al data lake en S3 → Glue → Athena.
Registro	Docker Hub (migrado de GHCR el 22-Set por problemas de visibilidad privada).

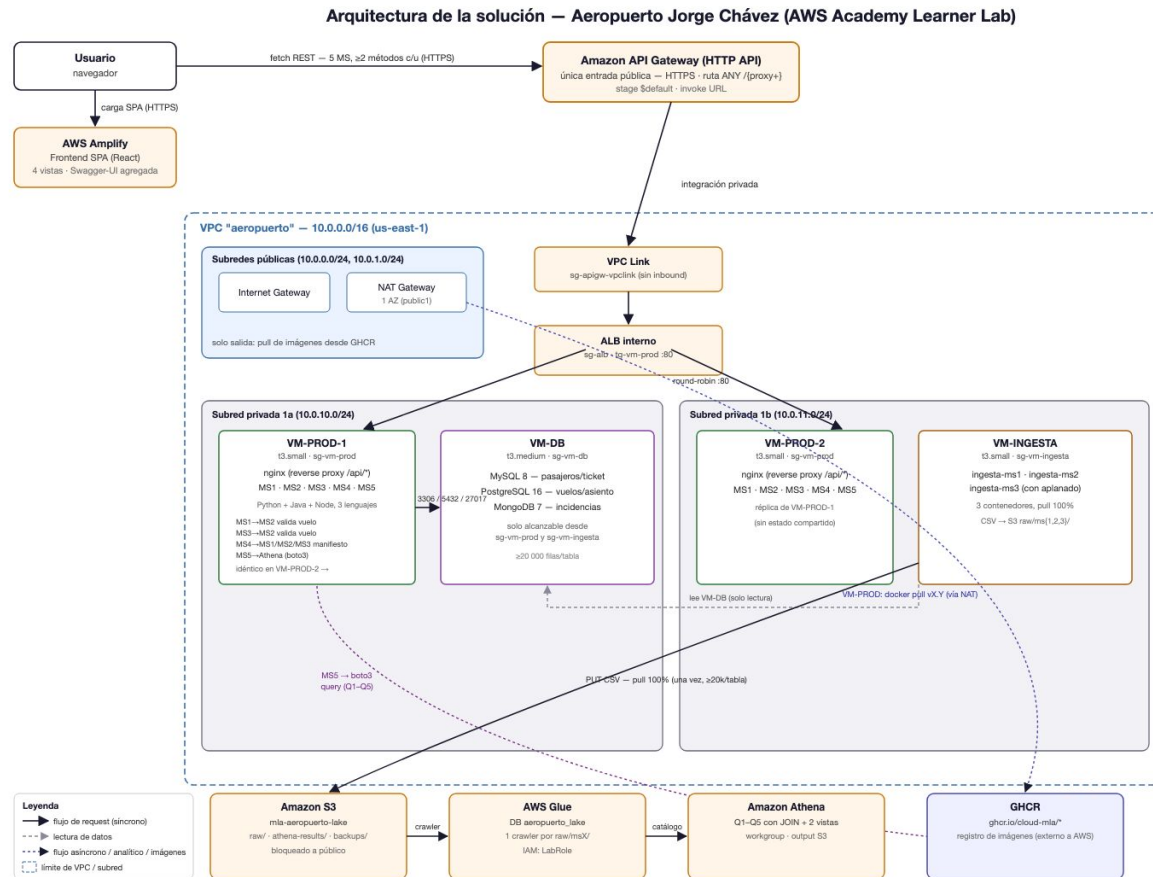
Diagrama v1 — topología validada en Learner Lab



Qué se validó en el Hito 1

- VPC, 4 EC2, ALB interno y API Gateway + VPC Link probados de punta a punta
- Boceto v0 → v1 tras validar el Learner Lab: fijar ALB vs NLB, Amplify vs CloudFront y VPC Link sí/no
- Sirvió como guía para el equipo y como material de la exposición virtual (no puntúa aún)

Diagrama v2 — arquitectura final desplegada



3 capas

Operación (request), persistencia e ingesta/analítica, con el sentido de cada flecha.

Agrupación

Subred pública/privada, leyenda y notas del Learner Lab.

Nota de registro

El diagrama v2 aún muestra GHCR; el registro vigente es Docker Hub (migración del 22-Set).

Flujo de una request: el navegador carga el SPA desde Amplify y lo su llama la API pública (HTTPS) directa a API Gateway. API Gateway cruza el VPC Link hacia un ALB interno que balancea entre las 2 VM-PROD (ingres + MS1+MS5); los microservicios leen sus 3 bases en VM-DB, todo eso aceptan conexión desde sg-vm-prd y sg-vm-ingesta. VM-INGESTA copia los datos de VM-DB a S3 (una vez, 220 000 files/table); Glue cataloga esos archivos y Athena corre las 2 vistas que MSX expose por REST. Las VM-PROD no tienen IP pública: su única salida a Internet es el NAT Gateway, usado para el docker `pull` de las imágenes publicadas en GHCR. Estado objetivo tras cerrar sg-vm-prd (BE-INT-06) — ver nota de despliegue temporal en docs/plant/personas/benja.a.md.

Decisiones de infraestructura

Security Groups

sg-vm-prod solo acepta tráfico de sg-alb — sin puertos abiertos a 0.0.0.0/0. sg-vm-db solo acepta desde sg-vm-prod y sg-vm-ingesta.

Bucket S3 por CLI

Gestionado por CLI, no por Terraform: una SCP de organización del Learner Lab bloqueaba el provider al leer Object Lock.

Terraform reproducible

100% reproducible: el despliegue final corre en una cuenta distinta a la original, sin rediseñar nada.

Sin IP pública

VM-DB y VM-INGESTA nunca tuvieron IP pública; la única entrada es API Gateway.

5 microservicios — de avance a completo

Microservicio	Stack · BD · Responsable	Hito 1	Hito 2
MS1 · Pasajeros / Tickets	Python + FastAPI · MySQL 8 · Guillermo	CRUD + 20k registros	✓ 25 000 tickets
MS2 · Vuelos / Operaciones	Java + Spring Boot · PostgreSQL 16 · Mariano	CRUD + modelo E/R + seed	✓ 20 500 vuelos
MS3 · Infraestructura / Incidencias	Node + Express · MongoDB 7 · Edinson	Endpoints de incidencias y recursos	✓ 25 000 incidencias
MS4 · Manifiesto (sin BD)	Python + FastAPI · agrega MS1+MS2+MS3 · Fabricio	Manifiesto consolidado vía asyncio	✓ Agrega MS1/MS2/MS3
MS5 · Analítico	Python + FastAPI + boto3 · Amazon Athena · Fabricio	Scaffold listo	✓ Consultas Athena

3 lenguajes, 3 motores de BD distintos — cumplido.

Consumo real entre microservicios

MS1 → MS2

POST /tickets valida el vuelo con GET /vuelos/{id}/exists

MS3 → MS2

POST /incidencias valida el vuelo antes de crear la incidencia

MS4 → MS1/MS2/MS3

GET /manifiesto/{vuelo_id} agrega y consolida la respuesta

Verificado 22-Set — log real de MS3

[MS3->MS2] GET http://ms2:8002/api/vuelos/999999/exists → 502 DEPENDENCIA_ERROR (contrato propio)

Contrato de error común (docs/contratos/errores.md) en las 5 APIs.

Carga masiva — evidencia COUNT(*)

25 000

tickets

MySQL (MS1)

184 394 filas totales cargadas

20 500

vuelos

PostgreSQL (MS2)

21 000 asientos

25 000

incidencias

MongoDB (MS3)

500 recursos · 40 000 asignaciones

Las 3 BD verificadas con COUNT(*) / countDocuments() independiente, 23-Set-2026. En el Hito 1 MS1 ya tenía 20k cargados; MS2 y MS3 se completaron en el Hito 2.

De la base de datos monolítica a 3 fuentes operacionales

Modelo repartido

El modelo monolítico de Base de Datos I pasa a 3 fuentes operacionales independientes.

Persona partida

Identidad base + extensión por rol (pasajero, empleado) según el microservicio dueño.

Referencias suaves

Las FK reales entre servicios pasan a “referencias suaves” validadas por HTTP, no por el motor de BD.

Detalle completo en [informe/latex/secciones/04-transformaciones.tex](#)

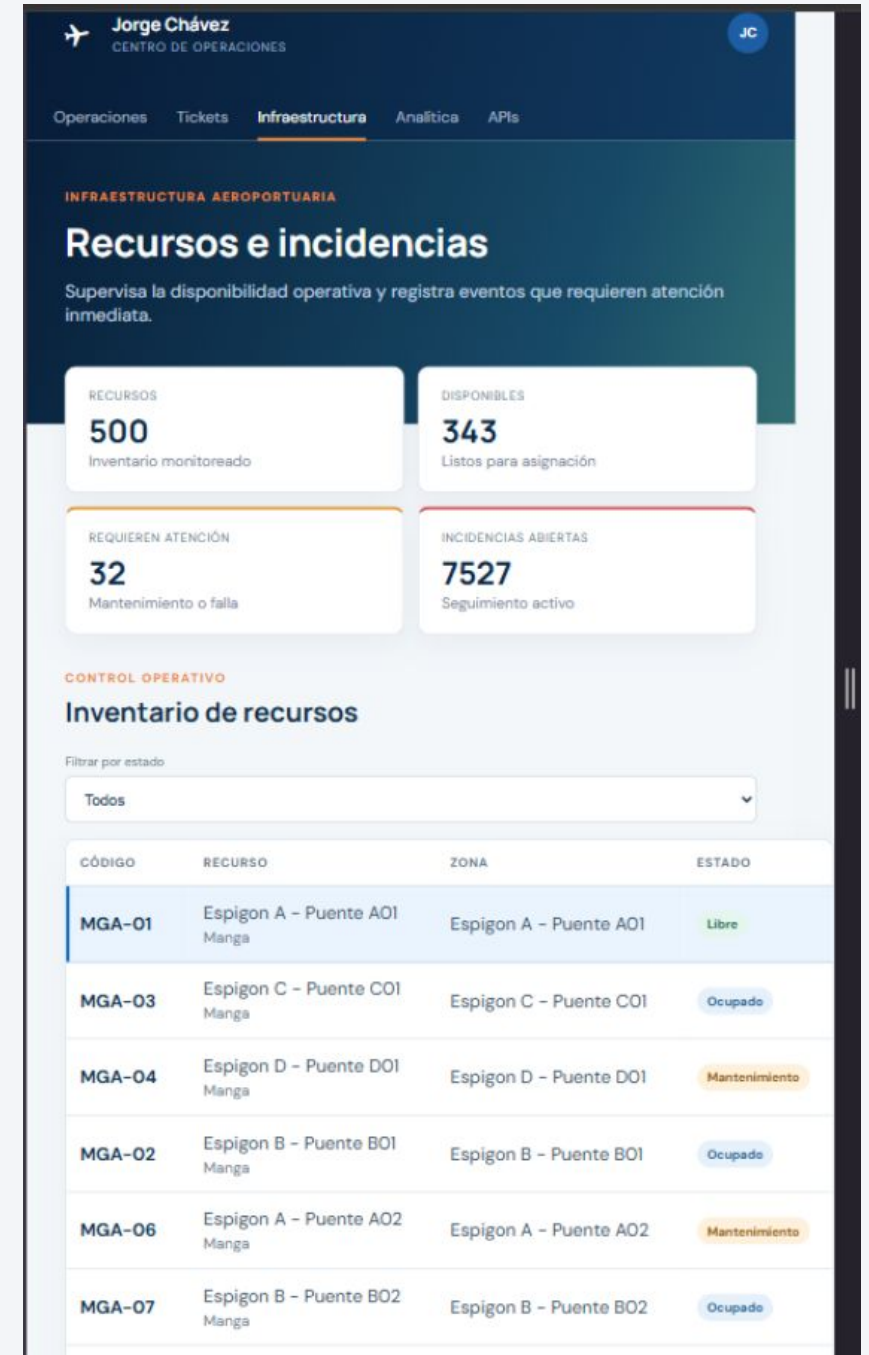
Frontend — React + Vite + TypeScript

Hito 1

- Integración real con MS2 (≥ 2 métodos REST)
- Proxy configurable por servicio (mocks $\leftrightarrow$ real)
- Manejo de errores por dependencia caída

Hito 2

- 4 vistas: consulta de vuelo/manifiesto · emisión de ticket/check-in · recursos e incidencias · dashboard analítico
- Matriz de cobertura REST: los 5 microservicios, ≥ 2 métodos por servicio
- Amplify bloqueado por política de la cuenta AWS — pendiente confirmar desde consola web o activar contingencia S3 + CloudFront



Ingesta de datos hacia el data lake

Hito 1

- VM-INGESTA + bucket S3 con prefijos raw/ms1, raw/ms2, raw/ms3
- ingesta-ms1 (MySQL → CSV → S3) corrido con datos reales
- ingesta-ms2 e ingesta-ms3 en curso

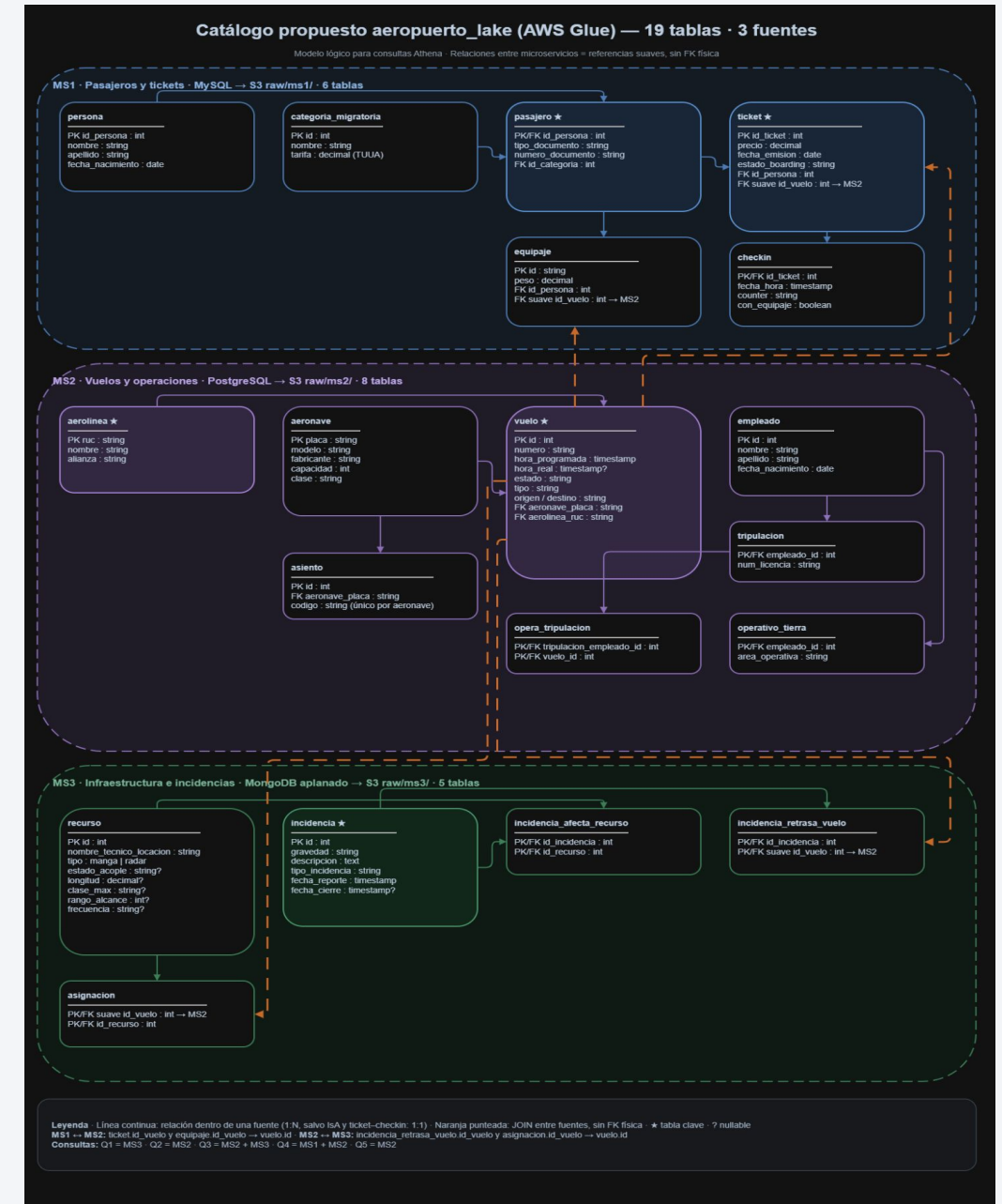
Hito 2

- 3 contenedores desplegados y corridos de verdad en VM-INGESTA contra la VM-DB real
- ingesta-ms3 aplanar incidencias.afecta_recursos / retrasa_vuelos a 2 tablas relacionales
- raw/ms1, raw/ms2 y raw/ms3 con datos reales — pull 100%



Catálogo Glue y consultas Athena

- Catálogo Glue aeropuerto_lake: 1 crawler por prefijo, 19 tablas catalogadas (las 3 fuentes completas)
- 5 de 5 consultas de negocio (Q1–Q5) con JOIN + 2 de 2 vistas, todas corriendo con datos reales



Despliegue y seguridad

2 VM-PROD

docker compose (nginx + 5 MS) detrás de un ALB interno

6 / 6 healthy

contenedores por VM con healthcheck activo, no solo “corriendo”

Backups

scripts/backup-db.sh probado con datos reales (MySQL 3.6 MB, Postgres 14 KB, Mongo 2 KB)

Smoke test

scripts/smoke-e2e.sh: público responde, privado bloqueado — en verde

Del Hito 1 al Hito 2: lo planeado vs lo entregado

Área	Meta al cierre del Hito 1	Resultado en el Hito 2	Estado
Backend	Healthchecks, carga $\geq 20k$ en MS2/MS3	25 000 / 20 500 / 25 000 filas verificadas; 6 contenedores healthy por VM; contrato de error común	✓ Cumplido
Frontend	Sumar MS1, MS3, MS4 y MS5 — 4 vistas completas	4 vistas, los 5 MS cubiertos con ≥ 2 métodos; deploy en Amplify bloqueado por política de cuenta	✓ Cumplido
Data Science	ingesta-ms2/ms3, catálogo Glue, ≥ 4 consultas + 2 vistas	3 ingestas, 19 tablas en Glue, 5 consultas + 2 vistas con datos reales	✓ Cumplido
Arquitectura	Diagrama final con nombres reales e IDs de SG	Diagrama v2 final incluido en el PPT; revisión cruzada DA-04 pendiente	✓ Cumplido
Entrega	Informe LaTeX, PPT de Hito 2, INDEX.md y demo ensayada	PPT e INDEX.md (9 repos) listos; informe en consolidación; ensayo de demo por agendar	✓ Cumplido

9 repos públicos en GitHub

cloud-computing-proyecto (docs)

ms1-pasajeros-api

ms2-vuelos-api

ms3-infra-api

ms4-manifiesto-api

ms5-analitica-api

aeropuerto-frontend

aeropuerto-data-science

aeropuerto-infra-deploy

Verificados con la API de GitHub (sin login). INDEX.md con las 9 URLs; ver también docs/plan/personas/ para el detalle de avance por persona.

Conclusiones y estado actual

Completo con evidencia real

Infra desplegada, API Gateway público, 3 BD con carga masiva $\geq 20k$, 3 contenedores de ingesta, MS3 $\rightarrow$ MS2 verificado. Pipeline completo: MySQL/PostgreSQL/MongoDB $\rightarrow$ S3 $\rightarrow$ Glue (19 tablas) $\rightarrow$ Athena (5 queries + 2 vistas).

Contingencias aplicadas

Docker Hub en vez de GHCR · bucket S3 gestionado por CLI · despliegue en cuenta AWS de respaldo.

Entrega: Hito 2 aplazado a Mar 23-Set 23:59 · exposición presencial y demo en vivo: Semana 7.

DEMO EN VIVO

Demo en vivo

- URL pública: API Gateway de producción

Gracias — preguntas

github.com/Cloud-MLA · Grupo Cloud-MLA — CS2032 Cloud Computing